Overview & Revision

- Overview
- Revision of C/C++ Programming

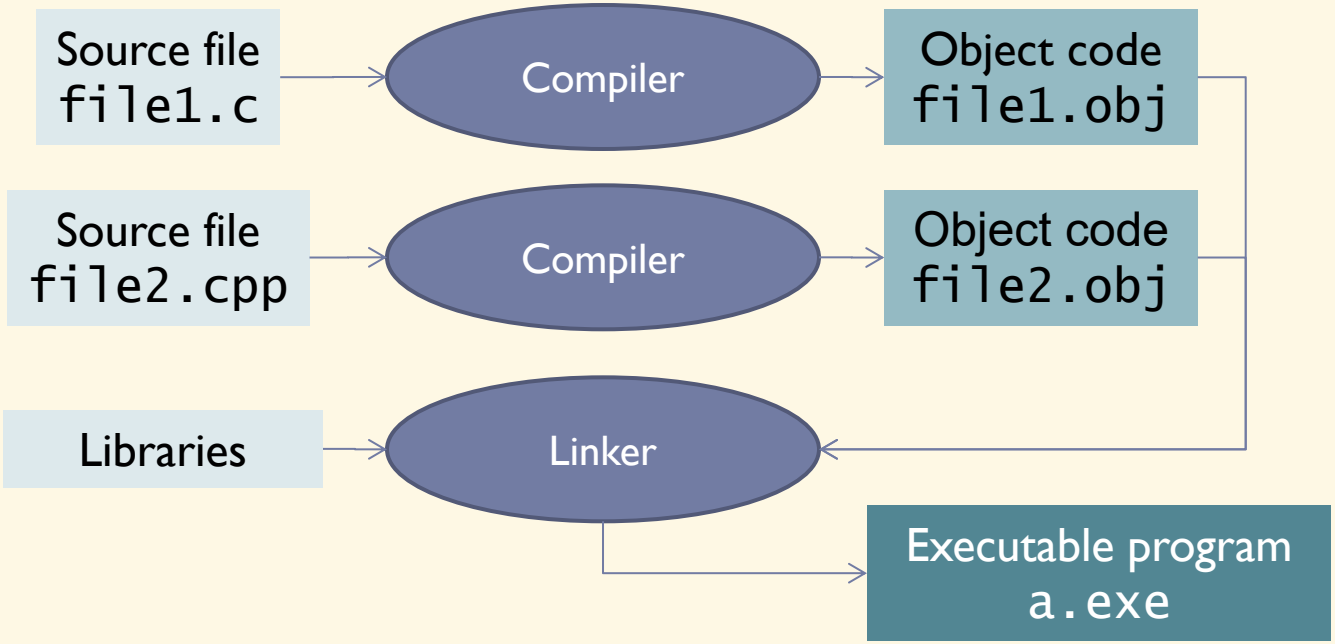
Overview

Introduction

- Data Structures & Algorithms (ET2100, ET2101E, ET2105E)
- Evaluation:
  - Midterm: In-class participation + Exercises + Homework
  - Final: Project
- Compilers: gcc/g++ or MS Visual C++
- Textbooks:
  - *Data Structures using C (2nd Ed.)*, R. Thareja, Oxford University Press, 2014
  - *Data Structures and Algorithm Analysis in C++ (4th Ed.)*, M. Weiss, Pearson, 2013
  - *C Programming – A Modern Approach (2nd Ed.)*, K.N. King, W.W. Norton & Company, 2008

Compiling a C/C++ program

- Compilation: is the process of converting a source code (written by human) into a program in the form of machine codes that can be executed



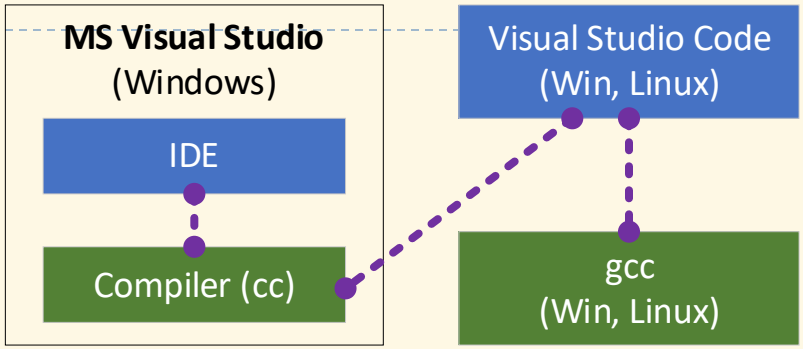
Compiling a C/C++ program *(cont.)*

- Benefits of compiling each source file individually:
  - Easy to divide and manage parts of the program
  - Only need to modify the involved files when something need to change

    → shorten maintenance, update time
  - Only need to recompile modified files instead of everything

    → shorten compilation time
- Modern compilers are able to optimize compiled codes (both data and instructions)
- Some popular compilers: MS Visual C++, gcc/g++, clang, Intel C++ Compiler, Watcom C/C++,…

Environment setup



- MS Visual C++:
  - https://visualstudio.microsoft.com/
- gcc under Linux (Ubuntu)
  - ```
    sudo apt update
    sudo apt install build-essential
    ```
- gcc under Windows
  - Option 1: Windows Subsystem for Linux (WSL) → Ubuntu for WSL
  - Option 2: Use a MinGW-w64, POSIX, ucrt build:
    - https://github.com/niXman/mingw-builds-binaries/releases
- Visual Studio Code:
  - https://code.visualstudio.com/
  - C/C++ Extension Pack
- OnlineGDB:
  - https://www.onlinegdb.com/
    ```

Command line compilation

- gcc/g++
  - Compile

    ```
    gcc -c <filename.c [...]>
    ```
  - Link

    ```
    gcc -o <filename> <filename.o [...]>
    ```

- Visual C++
  - Compile

    ```
    cl /c <filename.c [...]>
    ```
  - Link

    ```
    link <filename.obj [...]> /OUT:<filename.exe>
    ```

Useful GCC flags

`-Wall`	Turn on all warnings
`-c <filename>`	Compile to object file
`-o <filename>`	Link into output executable file
`-g`	Include debug information
`-I <folder>`	Use include folder
`-l <filename>`	Use static library

- Example:
  - `gcc -Wall hello.c -o runhello`
  - `./runhello`

Revision: C Programming

Display some text (export to screen)

- Syntax:
  - `printf("Format string", <values>);`
- Typing symbols:

Symbol	Type	Symbol	Type
%f, %e, %g, %lf	double, float	%x	int (hex)
%d	int	%o	int (oct)
%c	char	%u	unsigned int
%s	string of characters	%p	pointer

- Formatting:
  - `%[flags] [width] [.precision]type`
  - Ex: `%+15.5f`

Read user input (typed from keyboard)

- Syntax:
  - `scanf("Format string", <variable addresses>);`
- Examples:
  -
```
int age;
scanf("%d", &age);
```
  -
```
float weight;
scanf("%f", &weight);
```
  -
```
char name[20];
scanf("%s", name);
```

Variables and types

- Variables can hold values, which can be changed during runtime
- Variables need to be declared before using, with a type
- Global scope or limited within a function
- In standard C, internal variables need to be declared at the beginning of functions, before any instructions
- Variable declaration: <type> <list of variables>;
  - `int a, b, c;`
  - `unsigned char u;`
- Basic types:
  - `char, int, short, long`
  - `float, double`

Assignment

- To change the value of a variable with a new one
- Syntax:
  - `<variable> = <constant, variable> or <expression>;`
- Ex:
  - `count = 100;`
  - `value = cos(x);`
  - `i = i + 2;`
- Values of variables can be initialized at declaration (if not, the value is undefined):
  - `int count = 100;`
  - `char key = 'K';`

Constants

- Variables (yes, it's correct) whose values can not be changed at runtime
  - Declared by adding `const` keyword
  - Occupy memory (like any other variable)
- Ex:
  - `const double PI = 3.14159;`
  - `const char* name = "Nguyen Viet Tung";`
  - `PI = 3.14; /* error */`
- Another way to make a constant: using macro → not occupying memory (but un-typed)
  - `#define PI 3.14159`

Implementation dependency

- The size of basic types (`char`, `short`, `int`, `long`) are not exactly defined in the C standard, so they are implementation dependent
  - But the following points are always guaranteed:
    - size of `char` ≤ size of `short` ≤ size of `int` ≤ size of `long` ≤ size of `long long`
    - size of `char` = size of `unsigned char`, size of `short` = size of `unsigned short`, size of `int` = size of `unsigned int`,...
- If not specified, `short`, `int`, `long` and `long long` are signed by default, but `char` may not
  - `char` may be `signed` or `unsigned` depending on the compiler

Basic data (aka primitive) types

Type	Size (bytes)	Values
`char`	1	Character, integer
`int`	4	Integer
`short`	2	Integer
`long`	4	Integer
`long long`	8	Integer
`float`	4	Real (floating point)
`double`	8	Real (floating point)
`void`	0	Unspecified type

- Size of types may vary for machine and compiler, most commonly used is shown here
- Use `sizeof` operator to calculate the sizes of variables or data types in bytes:
  - `sizeof(x)`
  - `sizeof(int)`
  - `sizeof(15.6 * sin(y))`

Integer variable size and value range

- Signed and unsigned:

`signed char`	8 bits	–128 ÷ 127
`signed short`	16 bits	–32768 ÷ 32767
`signed int`	32 bits	–2147483648 ÷ 2147483648
`signed long`	32 bits	–2147483648 ÷ 2147483648
`signed long long`	64 bits	$-9.2\times10^{18} \div 9.2\times10^{18}$
`unsigned char`	8 bits	0 ÷ 255
`unsigned short`	16 bits	0 ÷ 65535
`unsigned int`	32 bits	0 ÷ 4294967295
`unsigned long`	32 bits	0 ÷ 4294967295
`signed long long`	64 bits	$0 \div 1.8\times10^{19}$

Numeric literals

- Integer literals

```
31          /* decimal int */
31U         /* unsigned int, or 31u */
037         /* octal */
0x1F        /* hexadecimal, or 0x1f */
31L         /* long */
31LU        /* unsigned long, or 31LU, 31ul, 31lu */
```

- Floating-point literals

```
123.4       /* double */
123.        /* double */
123.4F      /* float, or 123.4f */
123.F       /* float */
123.4L      /* long double */
1e-2        /* double */
123.4e-3    /* double */
.23e4       /* double */
.23e4F      /* float */
```

Character and string literals

- Character literals

```
'K'     /* normal character - 'K' */
'\113'  /* character given its octal code - 'K' */
'\x48'  /* character given its hexadecimal code - 'K' */
'\n'    /* newline character */
'\t'    /* tab character */
'\\'    /* backslash character */
'\"'    /* double quote character */
'\0'    /* null character (marks end of string)*/
```

  - Attn:
    - In C, character literals always have type of `int`, so `sizeof('c')` is same as `sizeof(int)`
    - In C++, character literals have type of `char`, so `sizeof('c')` is same as `sizeof(char)`

- String literals

```
"You have fifteen thousand new messages."
"I said, \"Crack, "  "we're under attack!\"."
```

Type casting

- Conversion of value of an expression from one type to another
- Implicit casting:
  - `float a = 30;`
  - `int b = 'a';`
- Explicit casting:
  - `int a = (int)5.6; /* take integer part */`
  - `float f = (float)1/3;`
- Not any type can be casted into any other:
  - `char* s = 2.3; /* not compiled */`
  - `int x = "7";  /* compiled in C but incorrect */`
  - `int y = 12 + "34";`

Integer promotions

- Integer types smaller than `int` are promoted in binary and tertiary operations (except shift operations) to avoid data lost
  - If all values of the original type can be represented as an `int`, the value of the smaller type is converted to an `int`; otherwise, it is converted to an `unsigned int`
  - ```
    signed char c1 = 100, c2 = 3, c3 = 4, r;
    r = c1 * c2 / c3;          /* 75 */
    ```
- Integer conversion rank:
  - Ranks in increasing order: `signed char`, `short`, `int`, `long`, `long long`
  - Rank of unsigned type equals rank of corresponding signed type
- Usual arithmetic conversions
  - Integer promotions are performed if necessary
  - Operand with the type of lesser rank is converted to the type of the operand with greater rank
    ```

Enumeration (enum)

- Used to define the possible values of a data type
- Syntax:
  - `enum <type name> { <values> };`
- Ex:
  - `enum WildAnimal {Tiger, Leopard, Bear, Deer };`
  - `enum WeekDay { Mon = 2, Tue, Wed, Thu, Fri, Sat, Sun = 1 };`
- Usage:
  - `enum WildAnimal wa = Tiger;`
  - `wa = Leopard;`
  - `enum WeekDay n = Thu;`

Defining new types from old ones (`typedef`)

- Used define new names for old types that are shorter or with different significations
- Syntax:
  - `typedef <original type> <new name>;`
- Ex:
  - `typedef double Height;`
  - `typedef unsigned char byte;`
  - `typedef enum WildAnimal WA;`
- Usage
  - `Height d = 165.5;`
  - `byte b = 30;`
  - `WA wa = Tiger;`

Revision questions

1. Is a constant a variable?
2. What are the results?

a)
```
int a = 10/3;
```

b)
```
double a = 10/3;
```

c)
```
short a = -10;
unsigned char b = 10;
unsigned int c = 10;
printf("%d %d", a < b, a < c);
```

d)
```
int a = -20;
unsigned int b = 10;
long long c = a + b;
printf("%d, %lld", a + b, c);
```

e)
```
signed char a = -1;
unsigned char b = a;
printf("%d", b);
```

f)
```
double x = 1.3, y = 1.6, z = 2.9;
printf("%d", z == (x + y));
```

Google C++ Style Guide(*): Avoid using unsigned types unless you have a special good reason to do so.

* Full version at: https://google.github.io/styleguide/cppguide.html

Exercises

- Write programs to:
  1. Calculate the sum of the digits of a given integer.
  2. Determine whether a number is prime.
  3. Print all the first n prime numbers.
  4. Calculate π using Monte Carlo method.

Structures, Arrays, Pointers & Strings in C

Structures (`struct`)

- Used to define data types composed of sub-variables (members, fields)
- Syntax:
  - `struct <data type> { <member variables> };`
- Ex:

```
struct Student {
    char name[20];
    int birth_year;
    int school_year;
};
```

- Usage:
  - `struct Student st = {"Le Duc Tho", 1984, 56};`
  - `st.birth_day = 1985;`
  - `st.school_year = 54;`

Structure initialization

- `struct Student s1 = {"Le Duc Tho", 1984, 2002};`

- `struct Student s2 = {"Le Duc Tho", 1984};`

- Designated initializers:

```
struct Student s3 = {
   .birth_year = 1984,
   .school_year = 2002,
   .name = "Le Duc Tho"
};
```

Unnamed structures

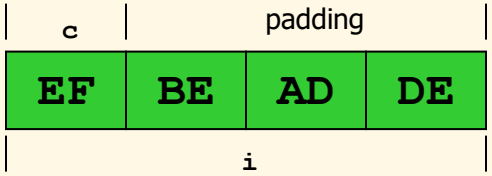
- Sometimes, struct name may be omitted:
  - ```
    struct {
       float x, y;
    } p1, p2, pa[3];
    ```
  - ```
    typedef struct {
       // …
    } Student;

    Student s1, s2;
    ```

Unions

- Example:
  - ```
    union AnElt {
        int   i;
        char  c;
    } elt1, elt2;

    elt1.i = 4;
    elt2.c = 'a';
    elt2.i = 0xDEADBEEF;
    ```
  - An element is an `int i` or a `char c`
  - Size of union = size of the largest field



- Example:
  - ```
    union color {
        struct {unsigned char R,G,B,A;} s_color;
        unsigned int i_color;
    };
    ```

Unions

- A union value itself doesn't "know" which case it contains

```
union AnElt {
   int   i;
   char  c;
} elt1, elt2;

elt1.i = 4;
elt2.c = 'a';
elt2.i = 0xDEADBEEF;

if (elt1 currently has a char) …
```

?

How should your program keep track whether `elt1`, `elt2` hold an `int` or a `char`?

?

Basic answer: Another variable holds that info

Tagged unions

- Tag every value with its case, i.e., pair the type info together with the union

Enum must be external to struct, so constants are globally visible

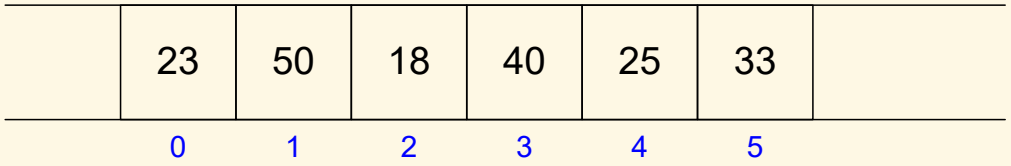
Struct field must be named

```
enum Union_Tag { IS_INT, IS_CHAR };

struct TaggedUnion {
   enum Union_Tag tag;
   union {
      int   i;
      char  c;
   } data;
};
```

Arrays

- Used to store multiple elements of a same type in memory at consecutive locations
- Are static pointers by nature
- Syntax:
  - `<type name> <variable name> [ <n° of elem.> ];`
- Ex:
  - `int age[6] = { 23, 50, 18, 40, 25, 33 };`



- Access elements: index starting from 0
  - `age[3] = 20;`
- Two-dimensional arrays (and multi-dimensional):
  - `float matrix[10][20];`
    `matrix[5][15] = 1.23;`

Array initialization

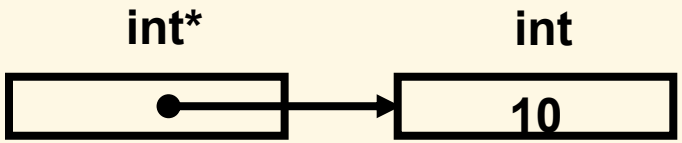
- `int a1[10] = {1, 2, 3, 4, 5};`
- `int a2[] = {1, 2, 3, 4, 5};`
- Designated initializers:
  `int a3[] = {1, 2, 3, [10] = 100, [14] = 200};`

Compound types

- Used to combine multiple sub-variables of same or different types
  - ```
    typedef struct {
      char name[20];
      unsigned int age;
      enum {Male, Female} sex;
      struct {
        char city[20];
        char street[20];
        int number;
      } address;
    } Student;
    ```
    ```

Pointers

- Pointer is variable whose value is address of some memory area of certain type
- Size of a pointer is same as that of `int`, but the size of memory area pointed by pointer is unknown (pointers do not have information about size)
- Declare pointers by * symbol:
  - `int *pInt;`
  - `char *pChar;`
  - `struct Student *pSt;`

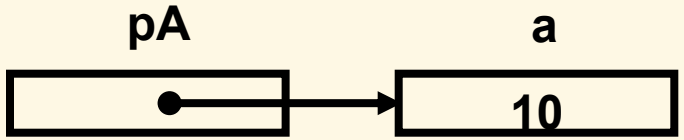


- Access to value (indirection) also by *:
  - `int aInt = *pInt;` (*pInt is an int that pInt points to)
  - `*pChar = 'A';`
  - `printf("Value: %d", *pInt);`

Changing address of pointers

- As value of a pointer is address, when changing its value, the pointer will point to another memory area
- Assign new address to a pointer by "=" operator as normal
  - `int *pInt2;`

    `pInt2 = pInt;`
- Address-of operator `&`: produces a pointer by taking address of a variable



  - `int a;`

    `int* pA = &a;  /* pA points to a */`
  - `&` is reversion of `*`: for any variable `a`, `*&a` is equivalent to `a`; and for any pointer `p`, `&*p` is equivalent to `p`

Illustration

```
char c = 'A';
int *pInt;
short s = 50;
int a = 10;

pInt = &a;
*pInt = 100;
```

Addresses of variables in memory here is increasing only for illustration. In reality, variables are allocated in stack from higher memory to lower → first variable has highest address

Address	1500	1501	1502	1503	1504	1505	1506	1507	1508	1509	1510	1511
Variable	char c	int* pInt				short s		int a				…
Value	'A'	1507				50		100				…

```
pInt:  1507
*pInt: 100
&a:    1507
a:     100
```

Void pointers (`void*`)

- Are pointers without type information
- Can be implicitly casted to any other pointer type, and vice versa (but not in C++)
  - ```
    void* pVoid;    int *pInt;     char *pChar;

    pInt = pVoid;           /* OK */
    pChar = pVoid;          /* OK */

    pVoid = pInt;           /* OK */
    pVoid = pChar;          /* OK */

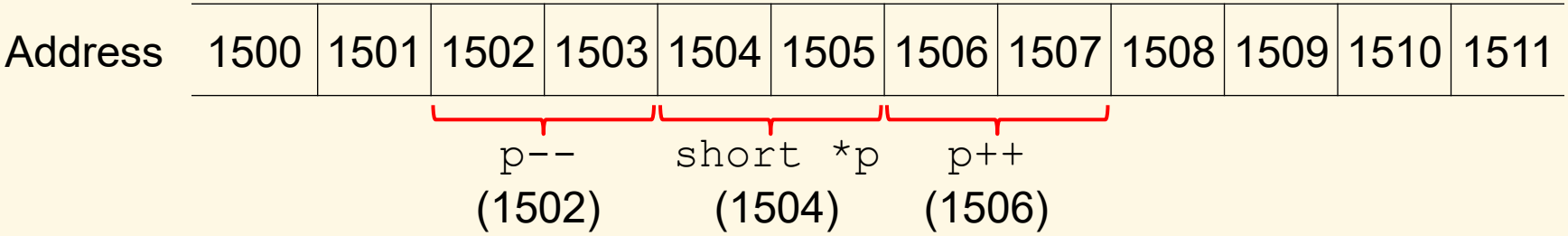
    pChar = pInt;           /* error */
    pChar = (char*)pInt;    /* OK */
    ```
- Indirection cannot be applied to `void*` pointers
  - ```
    *pVoid  /* error */
    ```
- `void*` pointers are used to access pure memory, or to manipulate variables of unknown type
  - ```
    memcpy(void* dest, const void* src, int size);
    ```
    ```

Null pointer (NULL)

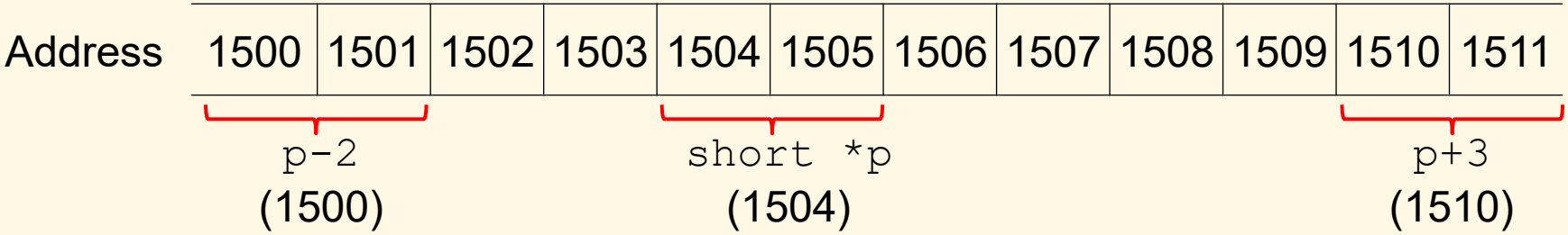
- Is a pointer constant with address of 0, type of void*, which signifies a pointer that does not point to any memory
- Is macro declared as follows technically:
  - `#define NULL   ((void*)0)`
- Impossible to assign values to null pointers
  - ```
    int *pInt = NULL
    *pInt = 100;    /* error */
    ```
- Distinguish null pointers with uninitialized ones (which point to random memory)
- Usually used to determine the validity of a pointer variable → to avoid errors, always assign NULL to pointer variables when unused
- As NULL is evaluated to 0, comparing a pointer to NULL can be ignored in logical experssions:
  - `if (p != NULL) …` → `if (p) …`
    ```

Pointer operations

- Increment/decrement: pointer points to the next/previous element (address increased/decreased by the size of the pointer type)



- Addition/subtraction: similarly



- Comparison: 2 pointers of same type can be compared using address like comparing two integers (greater, smaller, equal,...)
- A pointer can be subtracted to another of the same type

Pointers and arrays

- Array is static pointer (whose address is fixed), which points to (stores the address of) its first element
  - It is possible to manipulate arrays like pointers, except changing their address
  - ```
    int arr[] = {1, 2, 3, 4, 5};
    int x;
    *arr = 10;                  /* equivalent to: arr[0] = 10; */
    printf("%d", *(arr+2));     /* equivalent to: arr[2] */
    arr = &x;                   /* error */
    ```
- A pointer can be used as array and be manipuated as array
  - ```
    int *p = arr;
    p[2] = 20;          /* same as: arr[2] = 20; */
    p = arr+2;          /* same as: p = &arr[2]; */
    p[0] = 30;          /* same as: arr[2] = 30; or: *p = 30; */
    ```
- Conclusion: pointers and arrays can be used interchangeably, depending on programmer's convenience

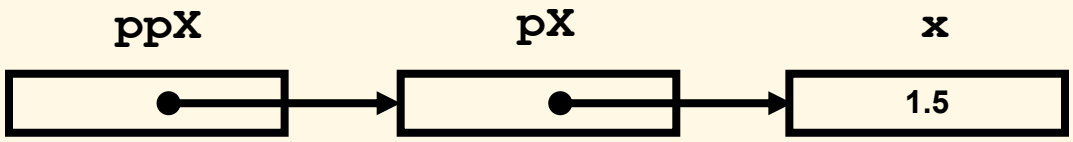
Pointers and arrays *(cont.)*

- Differences:
  - Impossible to assign new address to an array variable
  - Memory is allocated for array elements immediately at declaration (in stack)
  - `sizeof()` operator returns real size for arrays (total size of elements), while returns size of address for pointers
    - `float arr[5];` → sizeof(arr) returns 20 (5*4)
    - `float* p = arr;` → sizeof(p) returns 4 for 32-bit platforms
    - `sizeof(arr)/sizeof(arr[0])` → number of elements
- With pointer nature, it is possible to use negative index for arrays:

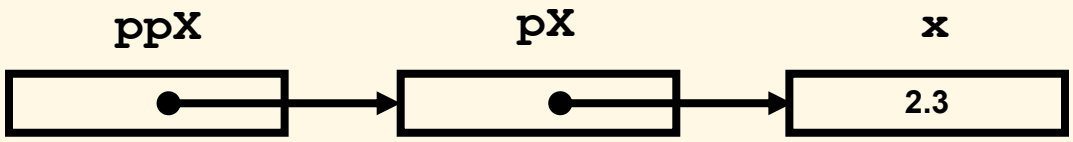
```
int arr[] = {1, 2, 3, 4, 5};
int *p = arr + 2;
p[-1] = 10;    /* same as: arr[1] = 10; */
```

Pointers to pointers

- It is possible to declare pointers to pointers in C
  - ```
    float x = 1.5;
    float *pX = &x;
    float **ppX = &pX;
    printf("%f", **ppX);   /* output: 1.5 */
    ```




```
**ppX = 2.3;
```



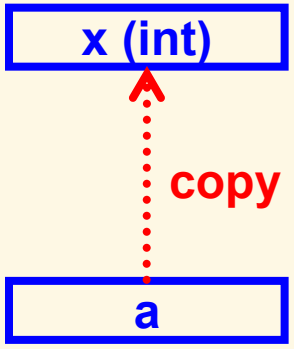
- Can be considered as 2-dimensional arrays, arrays of arrays, pointers to arrays, or arrays of pointers
```

Pointers to struct, union

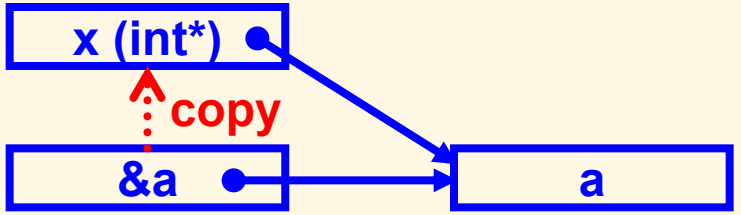
- For a pointer to struct or union, possible to use "->" operator to access member variables instead of "*" and "."
  - `p->member` is equivalent to `(*p).member`
- Ex:
  - ```
    typedef struct {
       int x, y;
    } Point;
    Point *pP = //...
    pP->x = 5;   /* same as: (*pP).x = 5; */
    (*pP).y = 7; /* same as: pP->y = 7; */
    ```
    ```

Passing parameters by value and by reference

- Function parameters are temporary variables created on function calls and destroyed when functions end → assigning values to parameters does not have effect on original variables
  - ```
    void assign10(int x) { x = 10; }
    int main() {
       int a = 20;
       assign10(a);
       printf("a = %d", a);
       return 0;
    }
    ```



- User pointers if desire to modify value of original variables
  - ```
    void assign10(int *x)
          { *x = 10; }
    int a = 20;
    assign10(&a);
    ```



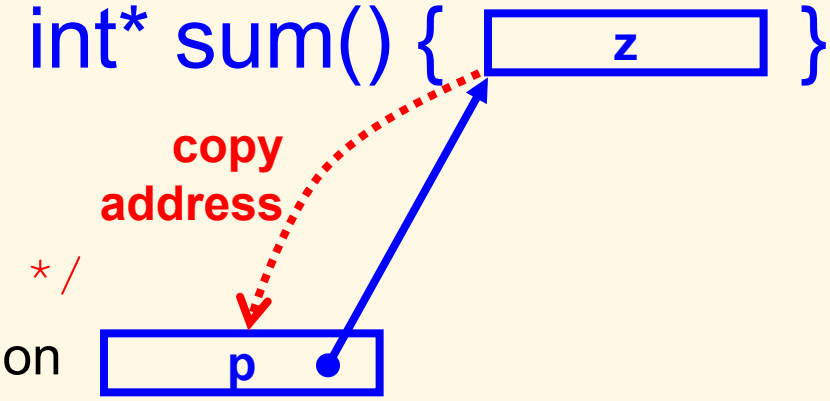
- Pointer parameters are often used as an alternative way to return values to callers, as each function has only one real return value

Pointers returned from functions

- Issue of returning address of local variables:

```
int* sum(int x, int y) {
    int z = x+y;
    return &z;
}

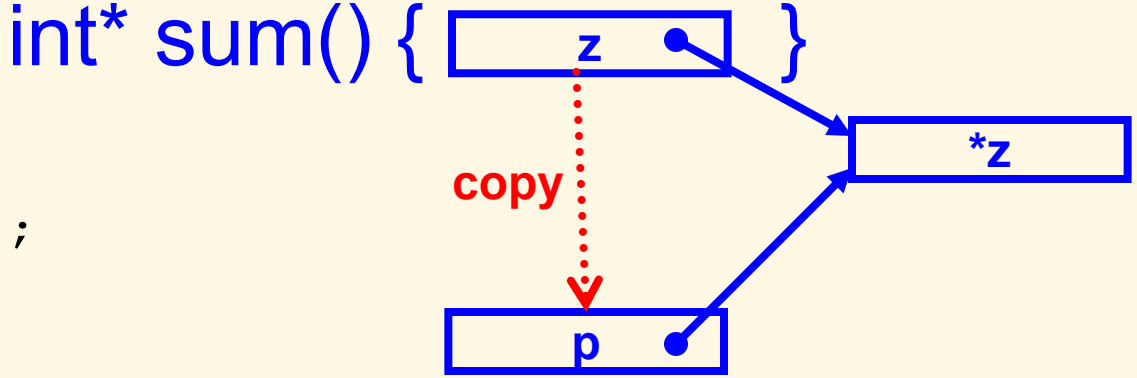
int* p = sum(2, 3); /* error */
```



- Solution: allocate memory inside function

```
int* sum(int x, int y) {
    int* z = (int*)malloc(sizeof(int));
    *z = x+y;
    return z;
}

int* p = sum(2, 3);
/* ... */
free(p);
```



Function pointers

- Are pointers to functions → a data type in C, usually used to call functions that are unspecified at writing time
  - `double (*SomeOpt)(double, double);`
  - `typedef double (*OptFunc)(double, double);`
    `OptFunc SomeOpt;`
- Ex:

```
double sum(double x, double y)  { return x+y; }
double prod(double x, double y) { return x*y; }
int main() {
    double (*SomeOpt)(double, double) = &sum;
    SomeOpt(2., 5.);    /* same as: sum(2., 5.); */
    SomeOpt = prod;
    (*SomeOpt)(2., 5.); /* same as: prod(2., 5.); */
    return 0;
}
```

- Attn: using & in assignments and * in calls is optional

Dynamic memory allocation

- Memory is allocated for variables at declaration (in stack)
- When memory need to be allocated on demand (size unknown at writing time) → dynamic allocation (in heap)

- Allocate memory:
  - `#include <stdlib.h>`
  - `void* malloc(int size) /* size: number of bytes */`
  - `int *p = (int*)malloc(10*sizeof(int)); /*alloc 10 int*/`

  - `void* calloc(int num_elem, int elem_size)`
  - `void* realloc(void* ptr, int size)`
  - If allocation fails, returns NULL → need to check
- Release (deallocate) an allocated memory:
  - `void free(void* p);`
  - `free(p);`

Conclusion on pointers

- Pointer is a powerful notion of C compared to other languages, but is a double blade knife, as it is hard to debug pointer related errors → programmers must master and use pointers flexibly

- Pointers are usually used in following contexts:
  - Passing values from function to outside via parameters
  - Function pointers
  - Data structures: linked list, queue, dynamic array,…

→ Will come back on related lessons

Strings of characters

- Are arrays of characters, terminated by '\0'
  - `char name[10] = "Tung";` (initialized by pointer)
  - `char name[10] = {'T', 'u', 'n', 'g', '\0' };` (by array)
  - `char *name = "Tung";` (pointer initialized by pointer)
  - `char *name = {'T', 'u', 'n', 'g', '\0' };` `/* error */`
- Ex: calculate length of string:
  - `for (n = 0; s[n]; n++) ;` `/* more preferable */`
  - `for (n = 0; *s; n++, s++) ;`
- Some popular string functions:
  - `#include <string.h>`
  - `int strlen(s)` → length of s
  - `char *strcpy(dst, src)` → copy string src to dst
  - `char *strcat(dst, src)` → concatenate src to the end of dst
  - `int strcmp(str1, str2)` → compare 2 strings, result: 1, 0, -1
  - `char *strstr(s1, s2)` → find position of s2 in s1

Exercise: String concatenation

- Implement a function `my_strcat(...)` to concatenate two strings given in parameter:
  - Input: two strings `s1` and `s2`
  - Output: a dynamically allocated string

- Example:
  - `my_strcat("hello_", "world!")`
  - Returns: `"hello_world!"`

Solution

```
char* my_strcat(const char* str1, const char* str2)
{
    int len1 = strlen(str1),
        len2 = strlen(str2);

    char* result = (char*)malloc((len1+len2+1) * sizeof(char));
    if (result == NULL) {
        printf("Memory allocation failure\n");
        return NULL;
    }

    strcpy(result, str1);
    strcpy(result + len1, str2);

    return result;
}
```

Question: Where comes `free(...)`?

Buffer overflow error

- Happens when program write more data than the size of buffer
  - Ex: copy a string of 10 characters into a buffer of 5 bytes

```
char s[5];
strcpy(s, "0123456789");  /* error */
```

- This error is dangerous as it will cause unpredictable effects, especially may help user to take ownership of computer then do anything → need to control the length of input data to fit into memory allocated to variables
- Standard C functions do not check for buffer overflow error

  → use extended functions available since C11

String and memory processing functions

- `memcpy_s(void* dest, int size, const void* src, int count);`
- `memmove_s(void* dest, int size, const void* src, int count);`

- `strcpy_s(char* dest, int size, const char* src);`
- `strcat_s(char* dest, int size, const char* src);`
- `_strlwr_s(char* str, int size);`
- `_strupr_s(char* str, int size);`

Data reading functions

- `gets_s(char* str, int size);`

- `scanf_s(const char* format, ...);`
  - Added buffer-size checking parameters
  -
```
int i;
float f;
char c;
char s[10];
scanf_s("%d %f %c %s", &i, &f, &c, 1, s, 10);
```
  - Similar to `fscanf_s()`, `sscanf_s()` functions

Revision questions

1. Are the following fragments working?

a)
```
char name[100];
scanf("%s", &name);
```

b)
```
char name[100];
char* name2 = name;
scanf("%s", &name2);
```

c)
```
char* name;
scanf("%s", &name);
```

d)
```
int* sum(int a, int b) {
  int c = a + b;
  return &c;
}
```

2. What are the results?

a)
```
int a[] = {0, 1, 2, 3, 4, 5, 6, 7, 8};
int* b = &a[1];
printf("%d, %d", b - a, (int)b - (int)a);
```

Exercises

- Write programs to:
  1. Convert a string into an integer number. Ex: "1234" → 1234.
  2. Convert an integer number into a string.
  3. Print a menu and ask the user to choose an option, then do a corresponding task. Use function pointers.

- What are the manners to return more than one values from a function?

Revision: C++ Programming

Overview

- Additional features compared to C:
  - Object-oriented programming (OOP)
  - Generic programming (template)
  - Many other small changes
- Small changes:
  - Conventional source files with .cpp extension
  - `main()` function can be declared with `void` return type:
    - `void main() { … }`
  - Use `//` to comment until end of line
    - `area = PI * r * r;  // PI = 3.14`
  - Built-in `bool` type and `false`, `true` boolean literals:
    - `bool b1 = true, b2 = false;`
  - Variables and constants in C++ can be declared anywhere in functions (not limited to function beginning only), even in `for` loops
  - New function-like syntax for type casting: `int(5.32)`
  - `enum`, `struct`, `union` keywords become optional in variable declaration

Some more significant features…

- Reference types: are pointers by nature
  - ```
    int a = 5;
    int& b = a;
    b = 10;    // → a = 10
    ```
  - ```
    int& foo(int& x)
      { x = 2; return x; }
    int y = 1;
    foo(y);            // → y = 2
    foo(y) = 3;        // → y = 3
    ```
- Namespace
  - ```
    namespace ABC {
       int x;
       int setX(int y)  { x = y; }
    }

    ABC::setX(20);
    int z = ABC::x;

    using namespace ABC;
    setX(40);
    ```
    ```

Some more significant features… *(cont.)*

- Dynamic memory allocation
  - Allocation: `new` and `new[]` operators
    - ```
      int* a = new int;
      float* b = new float(5.23f);
      long* c = new long[5];
      ```
  - Deallocation: `delete` and `delete[]` operators
    - ```
      delete a;
      delete[] c;
      ```
  - Attn: do not mix `malloc()`/`free()` and `new`/`delete`:
    - Memory allocated by `malloc()` must be deallocated by `free()`
    - Memory allocated by `new` must be deallocated by `delete`
- Function overload (functions with same name but different parameters):
  - ```
    int sum(int a, int b)                          {...}
    int sum(int a, int b, int c)                   {...}
    double sum(double a, double b)                 {...}
    double sum(double a, double b, double c)       {...}
    ```
- Exception handling `try ... catch`: self-study
    ```

Input and output

- Example:
  - ```
    #include <iostream>
    using namespace std;
    void main() {
       int n;
       cout << "Enter n: ";
       cin >> n;
       cout << "n = " << n << endl;
    }
    ```
- Input/output with C++:
  - Using iostream library
  - “cout <<” used for output to stdout (console by default)
  - “cin >>” used for input from stdin (keyboard by default)
  - Objects of C++ standard library defined in a namespace named `std`. If “using …” is not used, one must use std::cout, std::cin, std::endl instead
    ```

Smart pointers (since C++11)

- Abstract type to simulate pointers, while providing additional features:
  - Memory management
  - Bound checking
- Example:

```
shared_ptr<int> p1(new int(10));
shared_ptr<int> p2 = p1;
*p2 = 20;

// memory is reclaimed when
// the last reference is destroyed
```

More on C++ to revise

- Object and classes
  - Inheritance
  - Function & operator overload
- Generic programming with templates
- Modern C++
  - Move semantics
  - Lambda expressions
- Standard Template Library (STL)
- C++ Core Guidelines:
  - https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

Revision Questions

1. What is the equivalent mechanism for virtual methods in C?
2. Do the following fragments use a constructor or an assignment copy/casting operator?

   a)
   ```
   Cls o1;
   Cls o2 = o1;
   ```

   b)
   ```
   Cls o1, o2;
   o2 = o1;
   ```

3. Are the following fragments working correctly?

   a)
   ```
   Cls* p = (Cls*)malloc(3 * sizeof(Cls));
   ```

   b)
   ```
   Cls* p = new Cls;
   free(p);
   ```

Exercises

1. May the following code work? If yes, what is the condition?

```
Cls* x = nullptr;
x->f();
```

2. Write a program to: print a menu and ask the user to choose an option, then do a corresponding task.
3. Implement the `shared_ptr<T>` class by yourself.

Homework

Write programs to:

1. Read an integer N, allocate memory for N integers and input data, then print the array in reverse order
2. Read an array of floating-point numbers without knowing nor asking for the number of elements a priori (extend the array on demand)
3. Read a string $s1$, then copy it to another string $s2$ (without using strcpy nor memcpy)
4. Read two strings $s1$ and $s2$, then compare to see if they are equal (without using strcmp)
5. Read a string, then split it into an array of words
6. Declare two arrays of increasing `float` values, then merge them into another array also in increasing order